

SIT707 Software Quality and Testing

Credit Task: Unit Test Standards – AAA and FIRST

Name : Ayush Indapure

Student ID : 224880003

1. Overview

This report demonstrates improvements made to unit testing in the WeatherController project by applying:

- AAA (Arrange, Act, Assert) testing pattern
- FIRST principles (Fast, Independent, Repeatable, Self-validating, Timely)

The main issue addressed was non-deterministic behavior in the `testTemperaturePersist()` method due to dependency on system time.

Real-World Scenarios Where Unit Testing Standards Are Difficult as per previous feedback

In real software systems, it is often difficult to strictly follow AAA and FIRST principles due to external dependencies and non-deterministic behaviour.

1. Time-dependent systems (similar to this project)

Applications such as banking transactions, weather systems, and booking platforms rely on system time. If legitimate real system time is used directly in unit tests, it violates the Repeatable property of FIRST because execution time affects results. This makes tests flaky and unreliable unless time is controlled using techniques like Clock injection.

2. Database and network delays

In microservice-based systems, services depend on APIs and databases. These external calls introduce latency and unpredictability. This breaks the Fast and Independent properties of FIRST because:

- Tests are no longer fast due to network/database delays
- Tests are not independent because they rely on external systems

Mocking or stubbing is required to isolate unit tests.

3. Asynchronous processing systems

Systems using queues (e.g., Kafka(for observability in cloud), RabbitMQ) or background jobs cannot guarantee immediate execution. This makes it difficult to follow AAA strictly because:

- Act and Assert phases may not happen sequentially
 - Timing issues may cause inconsistent results
-

4. Randomized or AI-based logic

Systems using randomness (e.g., recommendation engines or AI models) produce different outputs for the same input. This violates Repeatable and Self-validating properties of FIRST unless randomness is controlled using seeded values or mocks.

2. Problem Identified

The original implementation of `persistTemperature()` used:

- `new Date()` to capture system time
- `Thread.sleep()` causing execution delay

Issue:

This caused inconsistency between expected and actual timestamps in unit tests due to execution delays.

Impact:

- Violates Repeatable property of FIRST
- Causes flaky unit tests

3. Solution Implemented

To resolve the issue, **Dependency Injection using Clock** was introduced.

Key Changes:

- Replaced system time (`new Date()`) with `Clock`
- Injected a fixed clock during testing
- Ensured deterministic timestamp generation

4. Updated WeatherController (Key Change)

Persist Method Fix:

```
String savedTime = sdf.format(  
    java.util.Date.from(Instant.now(clock))  
);
```

Added Clock Injection:

```
private Clock clock;  
  
public static WeatherController createWithClock(Clock clock) {  
    return new WeatherController(clock);  
}
```

5. Updated Test Case (AAA + FIRST)

testTemperaturePersist()

```
@Test  
public void testTemperaturePersist() {  
  
    // Arrange  
    Clock fixedClock = Clock.fixed(  
        Instant.parse("2026-04-28T10:15:30Z"),  
        ZoneId.systemDefault()  
    );  
  
    WeatherController testWC = WeatherController.createWithClock(fixedClock);
```

```
double temp = 25.0;
int hour = 1;

// Act
String savedTime = testWC.persistTemperature(hour, temp);

// Assert
assertEquals("10:15:30", savedTime);

testWC.close();
}
```

6. AAA Principle Application

Arrange

- Fixed Clock created
- Controller initialized
- Input values defined

Act

- `persistTemperature()` executed

Assert

- Timestamp validated against expected fixed time
-

7. FIRST Principles Analysis

Fast

- No real time dependency in test

Independent

- Test uses isolated controller instance

Repeatable

- Fixed clock ensures same output every run

Self-validating

- Uses JUnit assertions

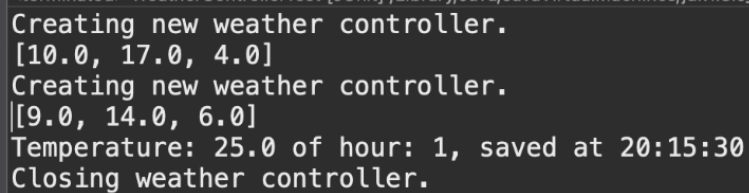
Timely

- Test added alongside feature improvement
-

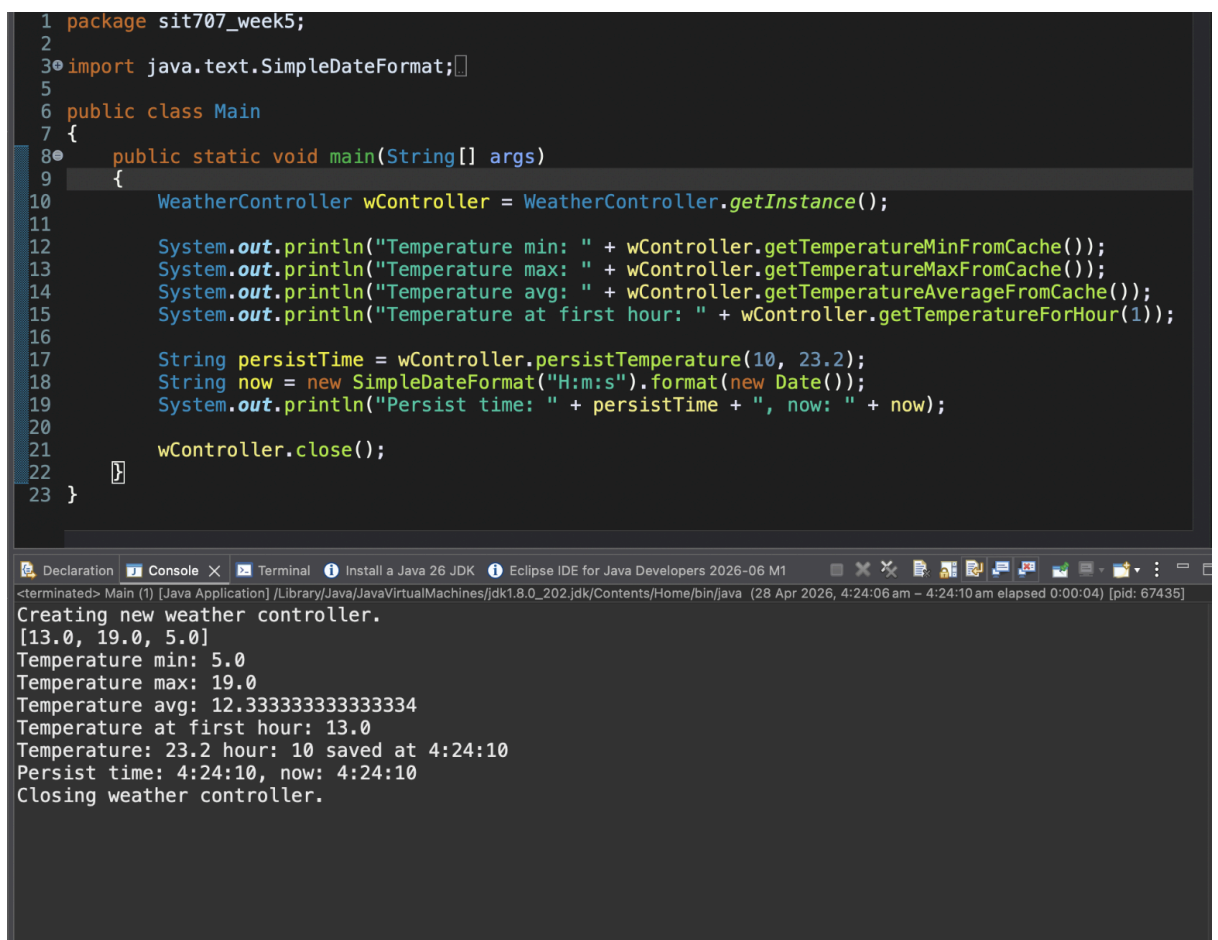
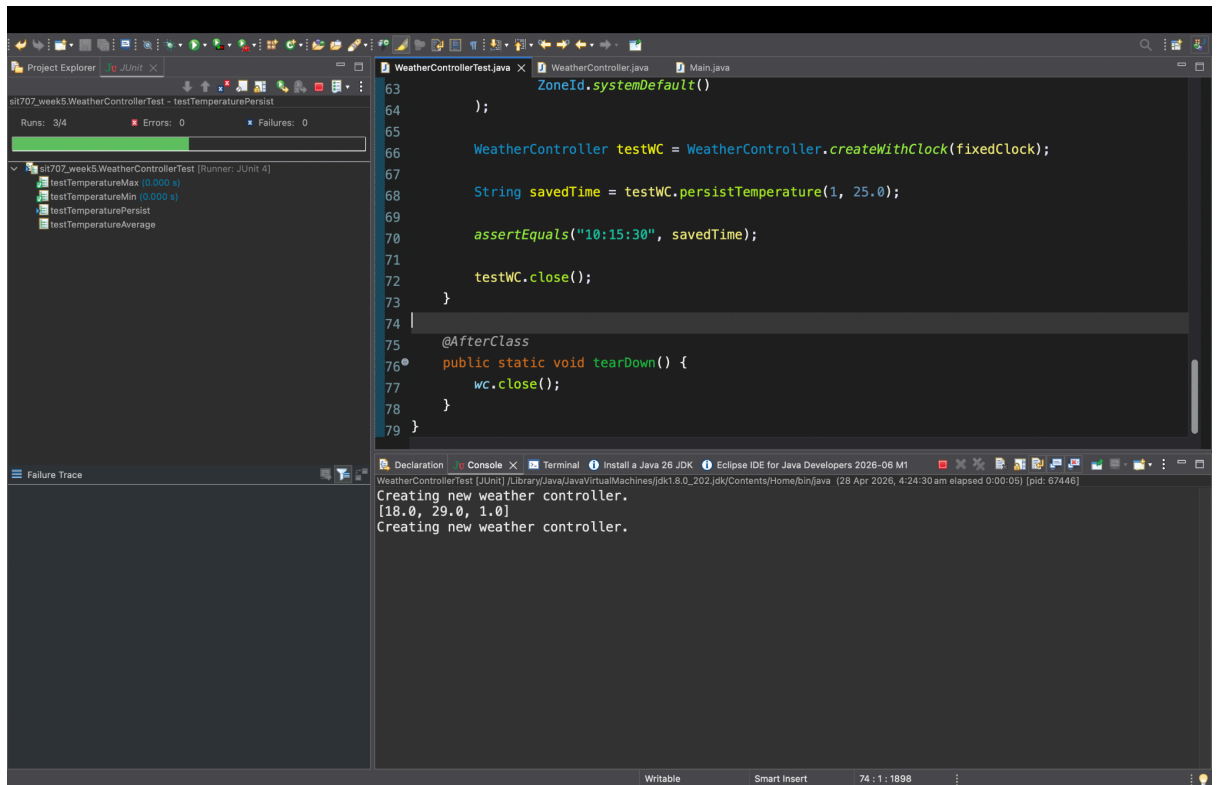
8. Conclusion

By introducing Clock dependency injection, the system was made deterministic and unit tests became reliable and repeatable. This ensures compliance with FIRST principles and improves overall test quality and maintainability.

9. Evidence

A terminal window with a dark background and light gray text. The text shows the execution of a test for a weather controller. It starts with 'Creating new weather controller.', followed by a list of values '[10.0, 17.0, 4.0]'. Then it says 'Creating new weather controller.' again, followed by '[9.0, 14.0, 6.0]'. The next line is 'Temperature: 25.0 of hour: 1, saved at 20:15:30'. The final line is 'Closing weather controller.'.

```
Creating new weather controller.  
[10.0, 17.0, 4.0]  
Creating new weather controller.  
[9.0, 14.0, 6.0]  
Temperature: 25.0 of hour: 1, saved at 20:15:30  
Closing weather controller.
```



- Screenshot of Eclipse console output
- Screenshot of GitHub repository with latest commit

10. Reflection

This task demonstrated the importance of avoiding system dependencies in unit tests. Real-world delays such as time and sleep calls should not be part of unit testing logic. Instead, controlled test environments should be used to ensure stability.